Marketing Automation Architecture Design

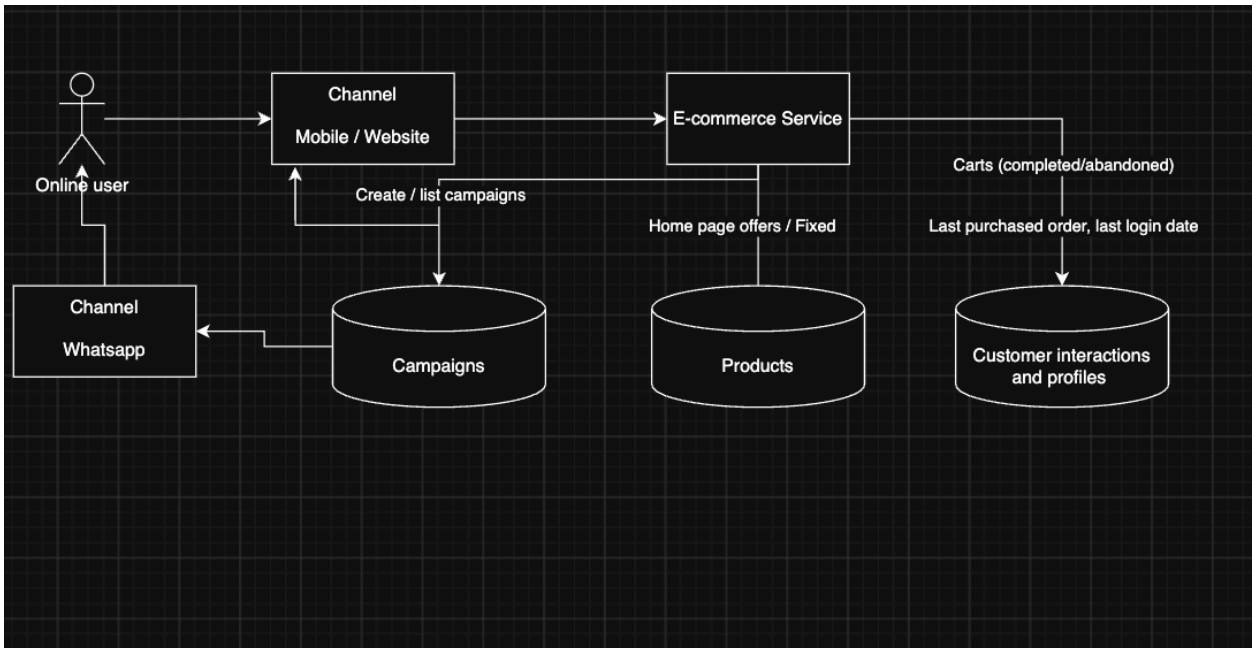
Problem statement/pain points

- Homepage / Products page Personalization. (feature stores)
    - New user: Cold start, Show static offers.
    - Personalize the home page offers per segment.
    - Personalize the home page offers per user.
- Next-best action
    - Abandon cart.
    - Potential churned customers.
- Customer care Chatbot, supported with RAG.
- Agentic Marketing - Campaign Automation.
    - Generate new marketing campaign by Agentic AI.

Goals/non-goals, Solution Assumptions

- Goals
    - Design personalization and Agentic Marketing system that should increase the CSAT to 80% after current 60%. By measuring customers rating and text feedback.
- Non-Goals
    - Lower refunds.
    -

As-is Architecture, Explore the Enterprise Architecture



Assumptions

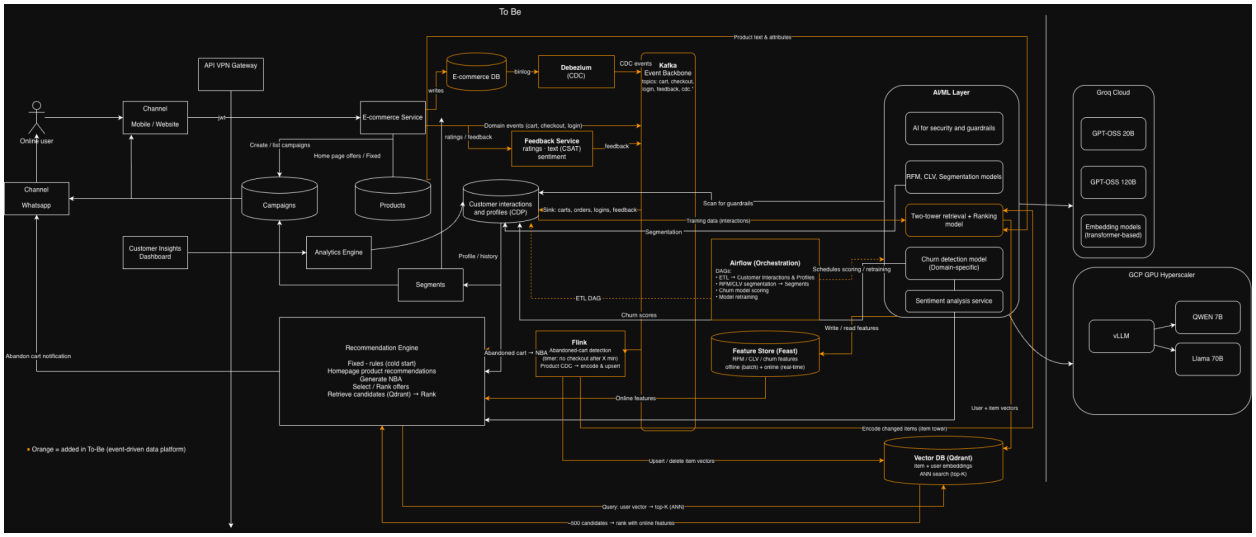
- No of concurrent users based on last 6 months (50 users).
- Seasonality (Ramadan, Holidays, …) Peak hours. 100 users.

Architecture principles, key decisions and trade-offs

- Data and compliance
    - Data should be stored in-country (on transit, on disk).
    - GDPR.
    - PDPL PII.
    - AI Guardrails to achieve clean text and limit the AI errors. Prompt injections.
- Security
    - All internal systems should be behind VPN.
    - All external systems should be authenticated and use https.

Context Viewpoint

Only Context



Components description (TBD)

- Component 1: describing it’s job, and major features it’s handling, also the major logical components inside, 2-3 lines per component.

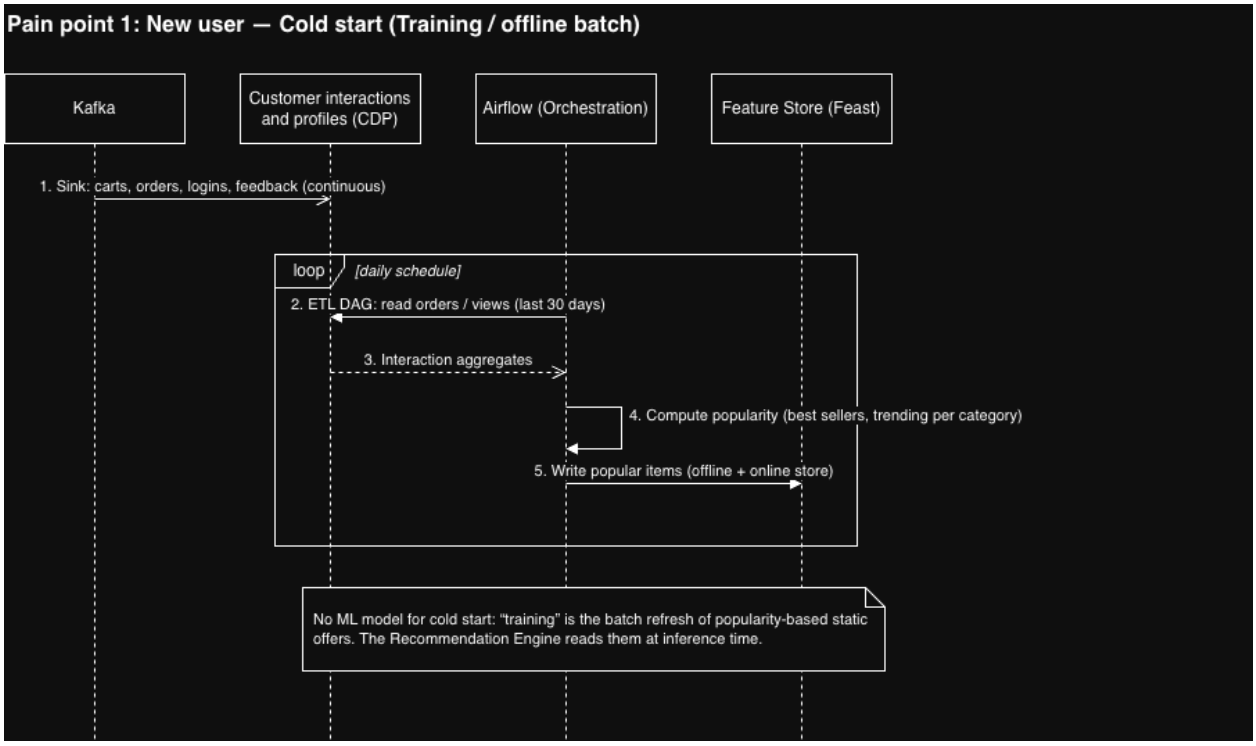
Components Viewpoint

(Ingestion/training, and inference flows for each use case)

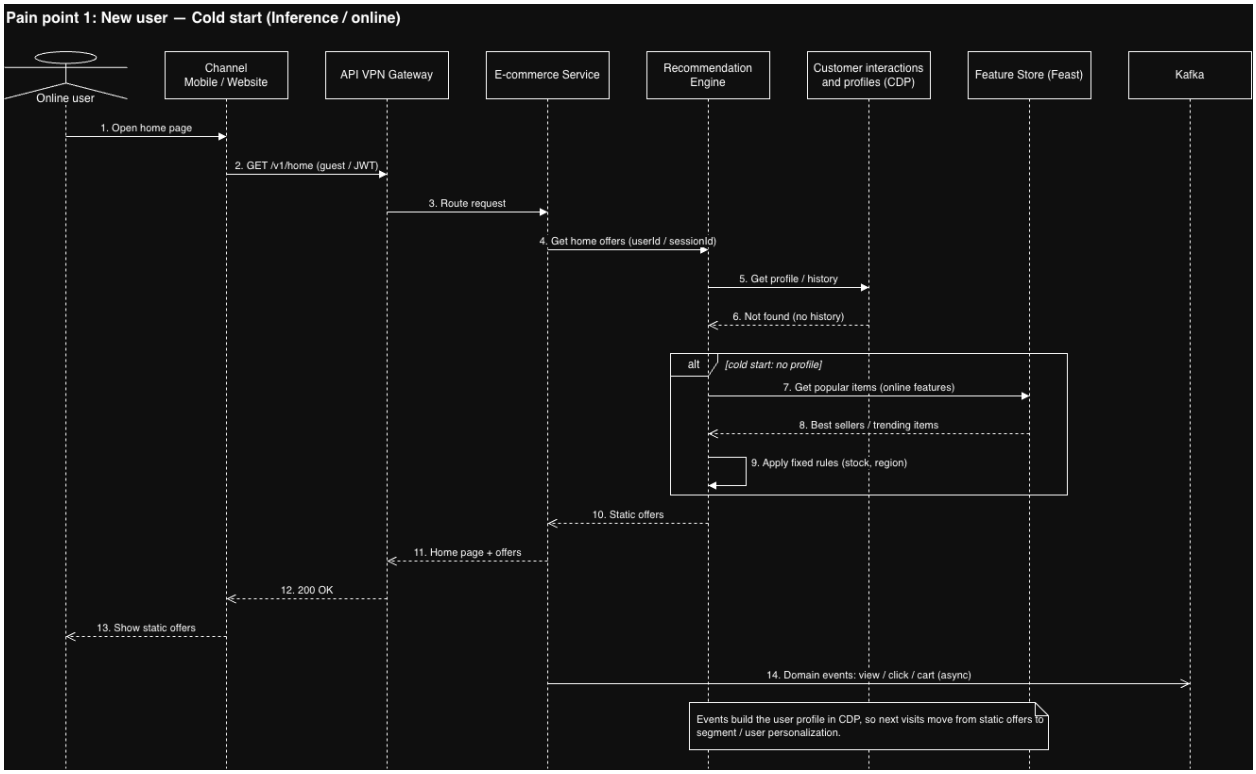
Homepage / Products page Personalization

New user: Cold start, Show static offers

Ingestion/Training flow

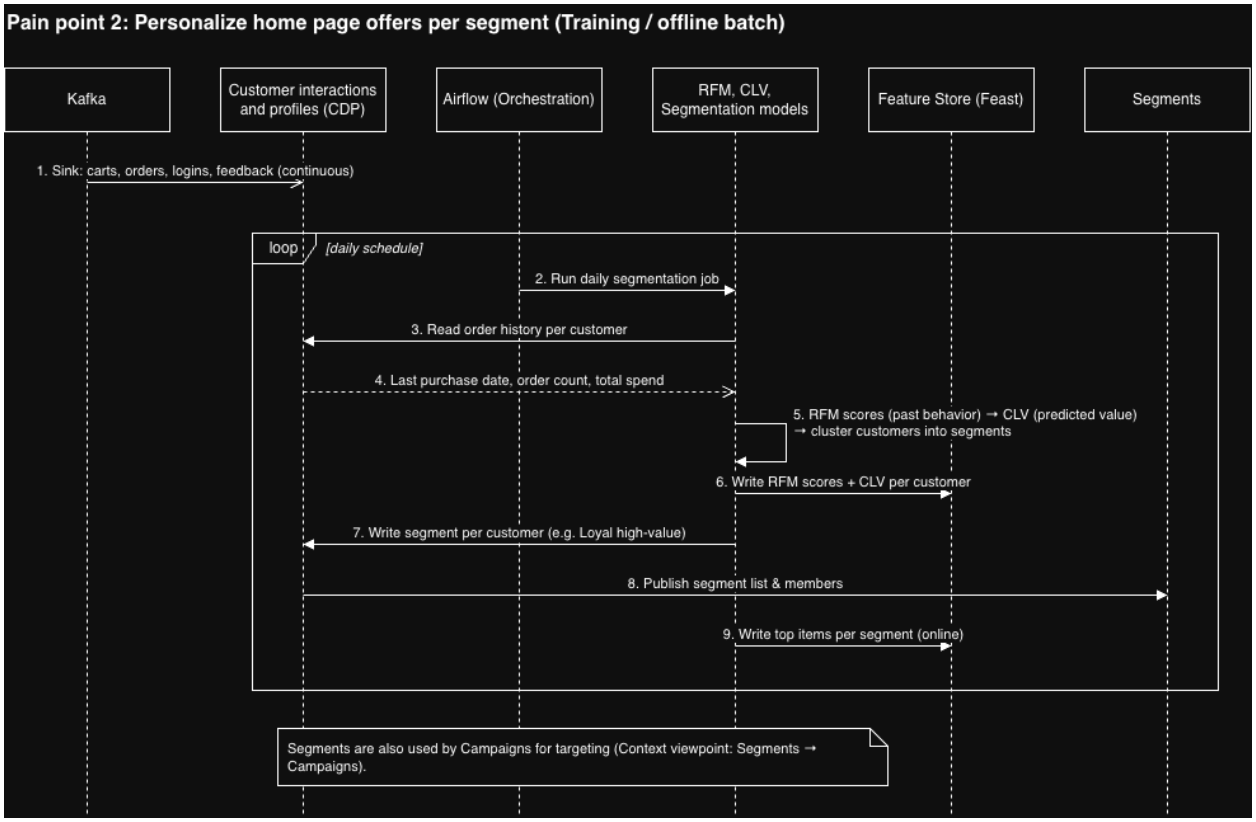


Inference

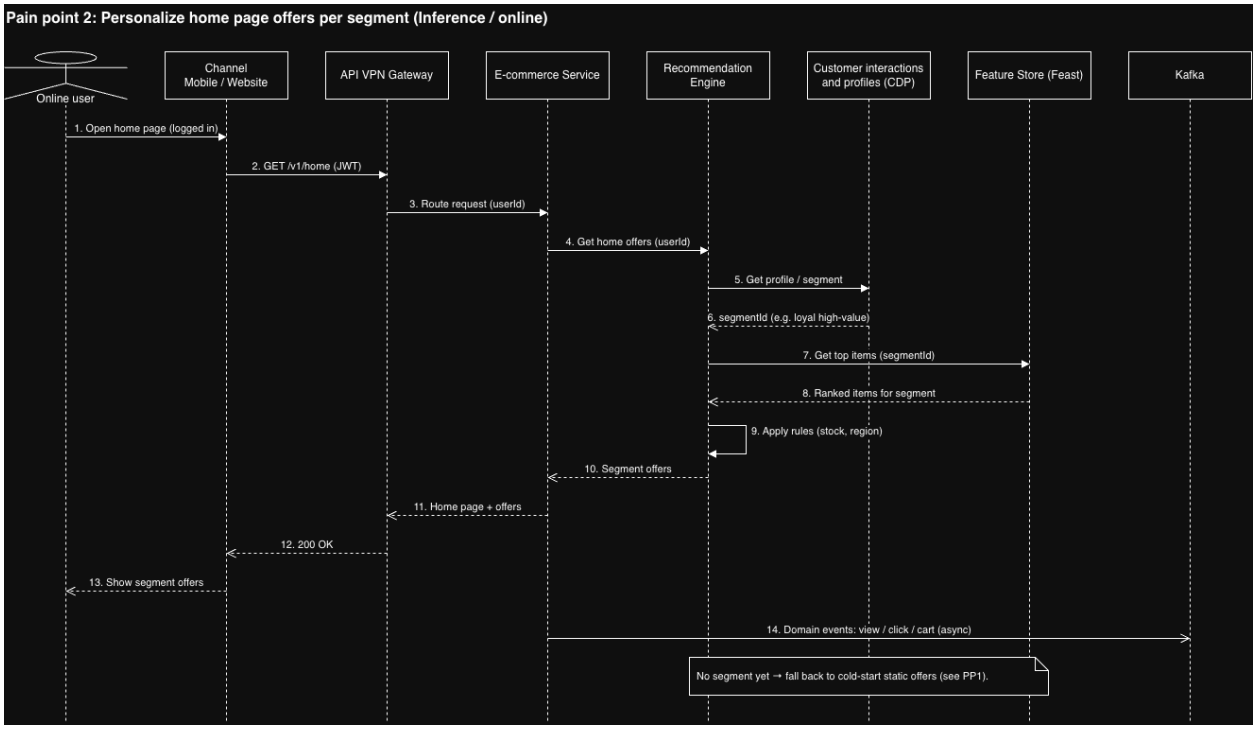


Personalize the home page offers per segment

Ingestion/Training flow

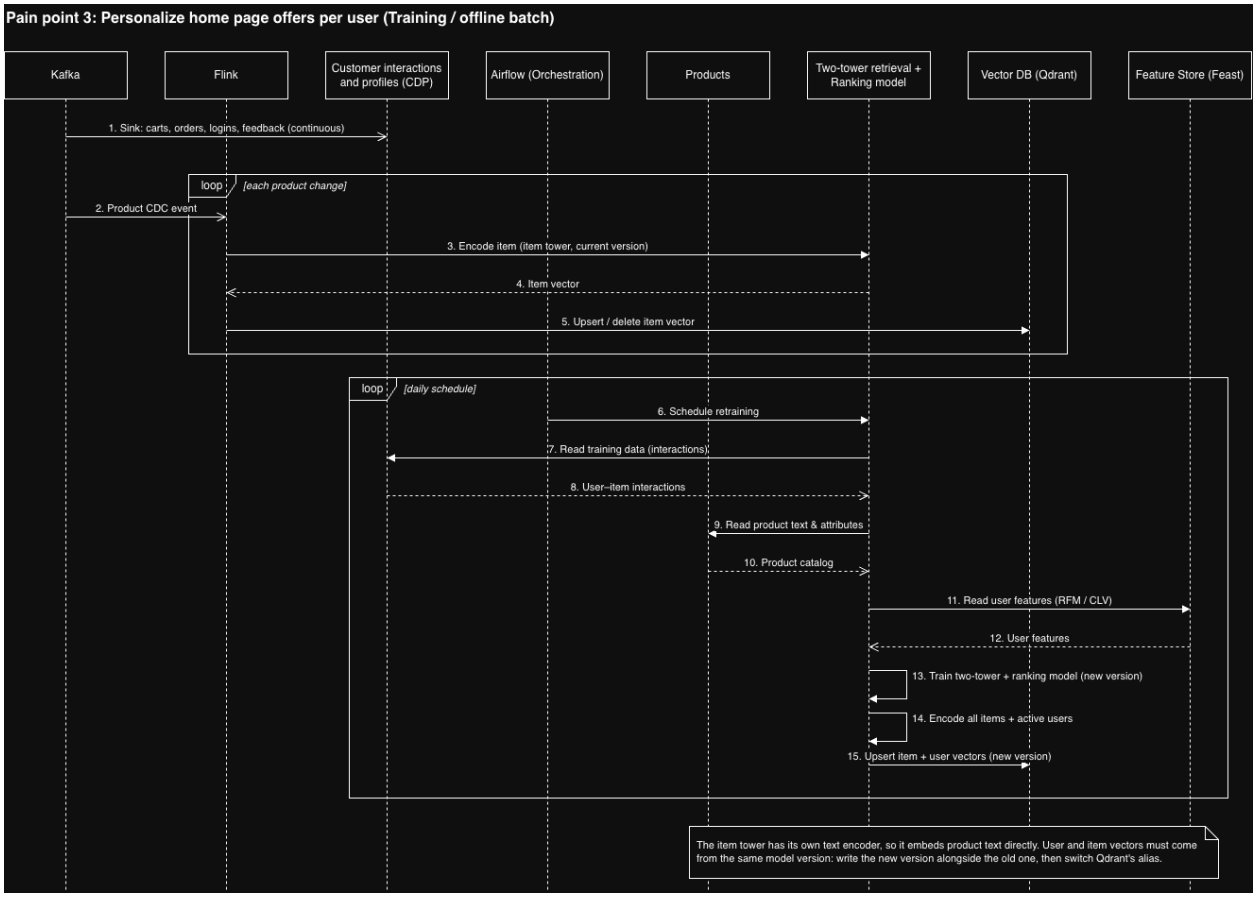

Pain point 2: Personalize home page offers per segment (Training / offline batch)
Kafka
Customer interactions and profiles (CDP)
Airflow (Orchestration)
RFM, CLV, Segmentation models
Feature Store (Feast)
Segments
1. Sink: carts, orders, logins, feedback (continuous)
loop
[daily schedule]
2. Run daily segmentation job
3. Read order history per customer
4. Last purchase date, order count, total spend
5. RFM scores (past behavior) → CLV (predicted value) → cluster customers into segments
6. Write RFM scores + CLV per customer
7. Write segment per customer (e.g. Loyal high-value)
8. Publish segment list & members
9. Write top items per segment (online)
Segments are also used by Campaigns for targeting (Context viewpoint: Segments → Campaigns).

Inference

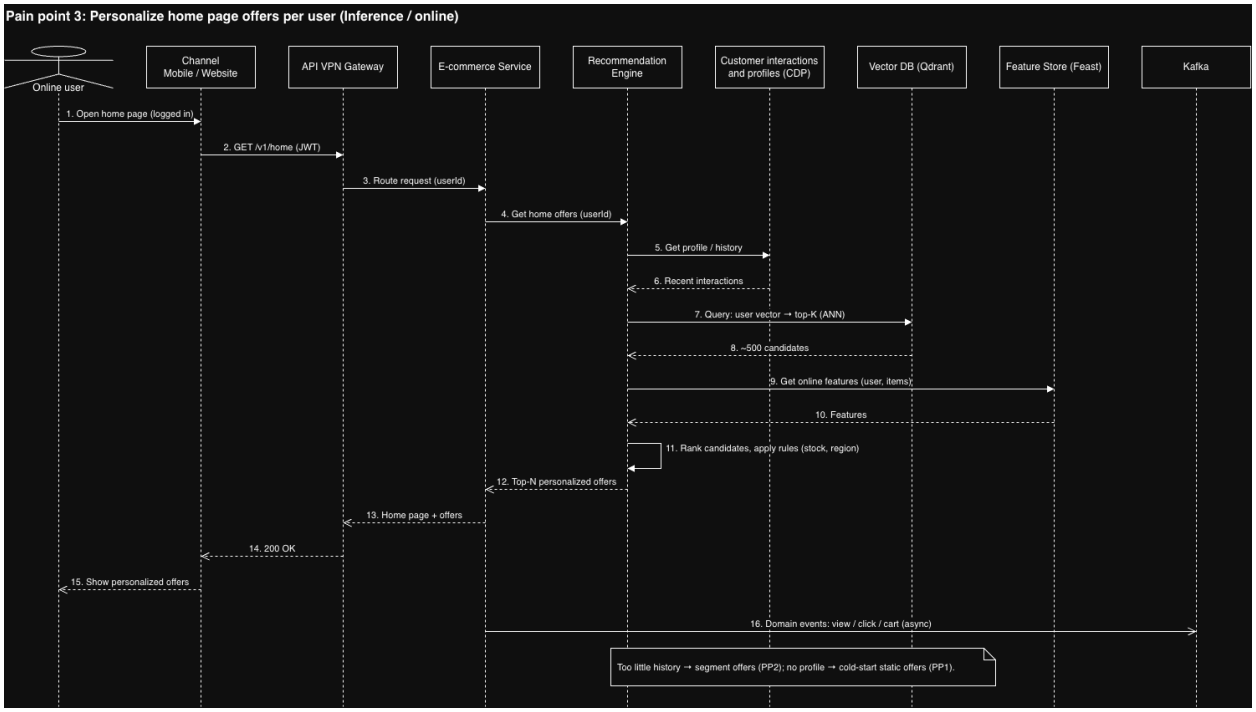

Pain point 2: Personalize home page offers per segment (Inference / online)
Online user
Channel Mobile / Website
API VPN Gateway
E-commerce Service
Recommendation Engine
Customer interactions and profiles (CDP)
Feature Store (Feast)
Kafka
1. Open home page (logged in)
2. GET /v1/home (JWT)
3. Route request (userId)
4. Get home offers (userId)
5. Get profile / segment
6. segmentId (e.g. loyal high-value)
7. Get top items (segmentId)
8. Ranked items for segment
9. Apply rules (stock, region)
10. Segment offers
11. Home page + offers
12. 200 OK
13. Show segment offers
14. Domain events: view / click / cart (async)
No segment yet → fall back to cold-start static offers (see PP1).

Personalize the home page offers per user

Ingestion/Training flow



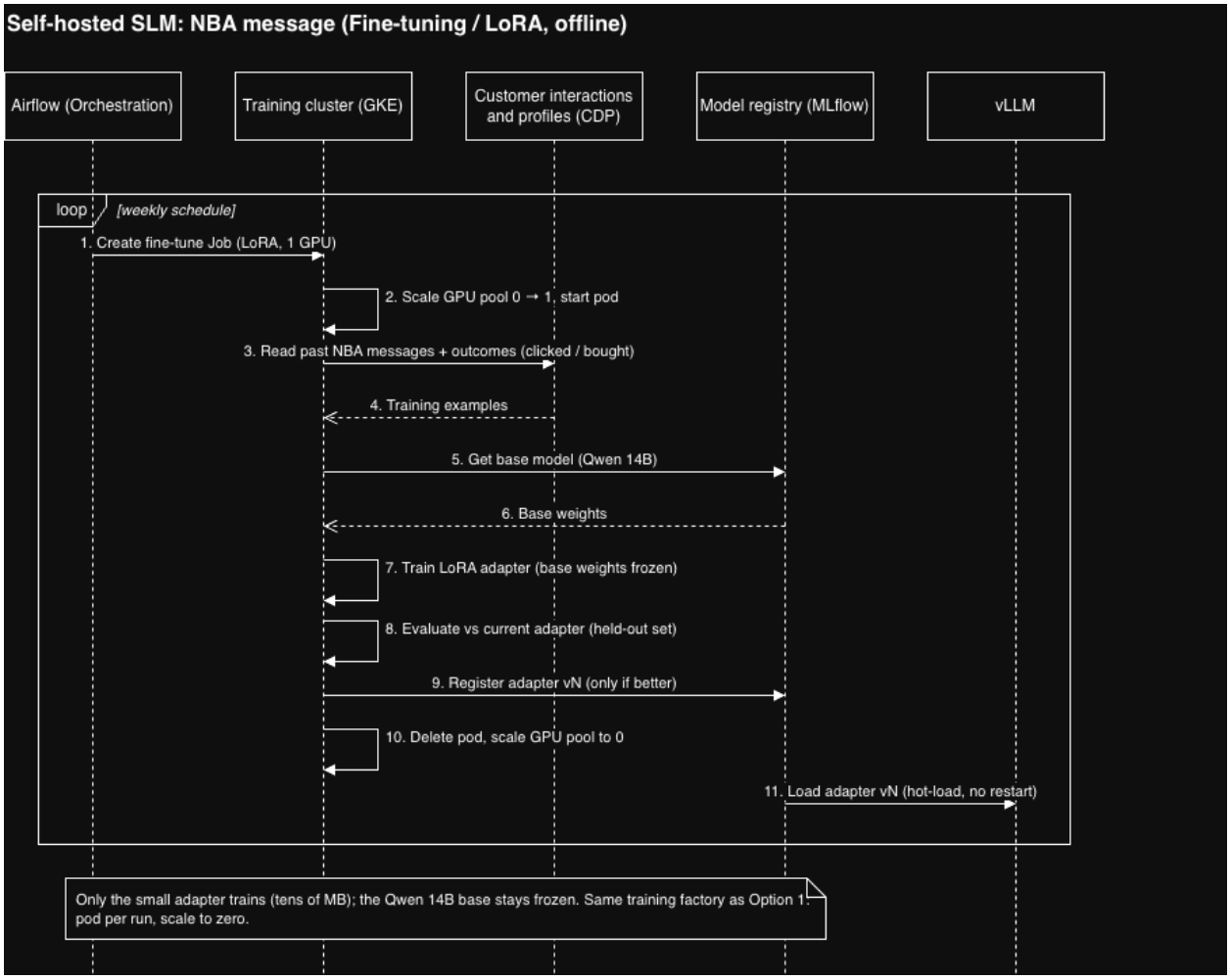
Inference



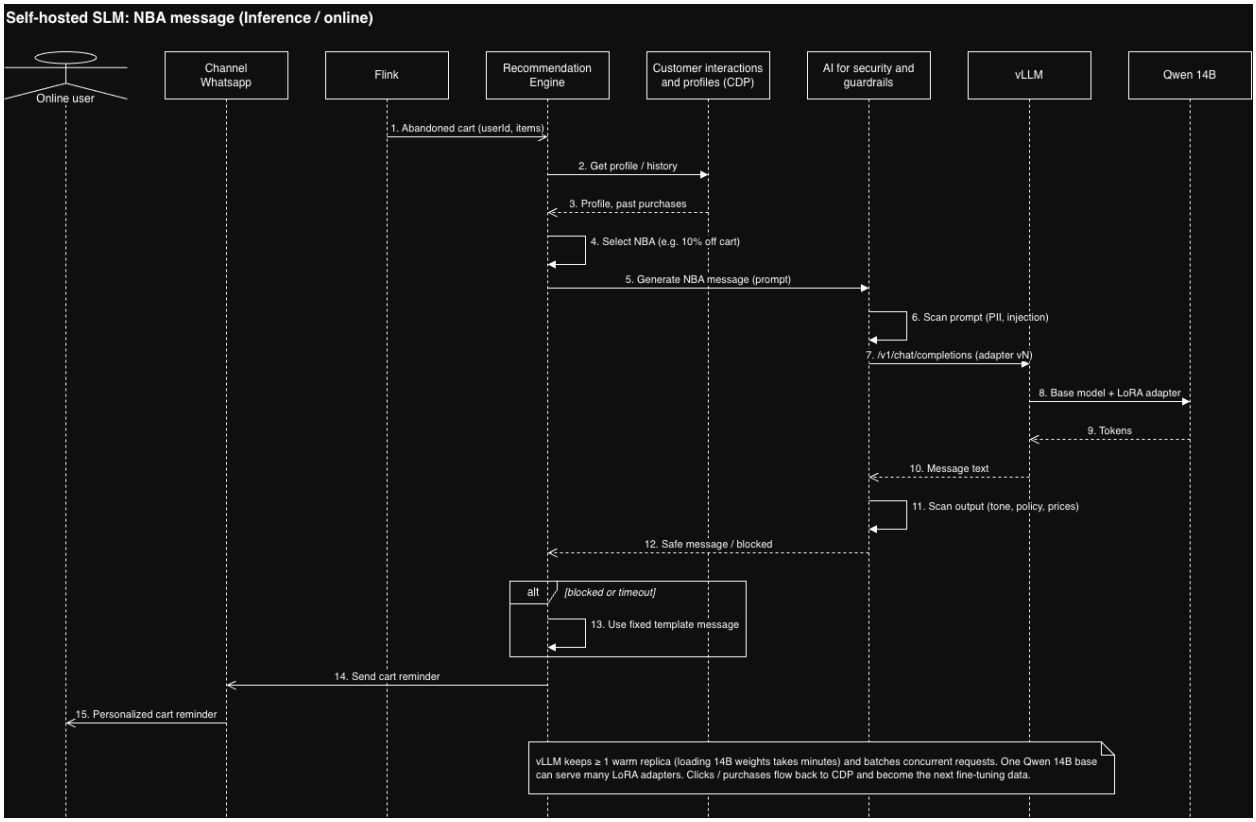
Next-best action

Abandon cart

Ingestion/Training flow

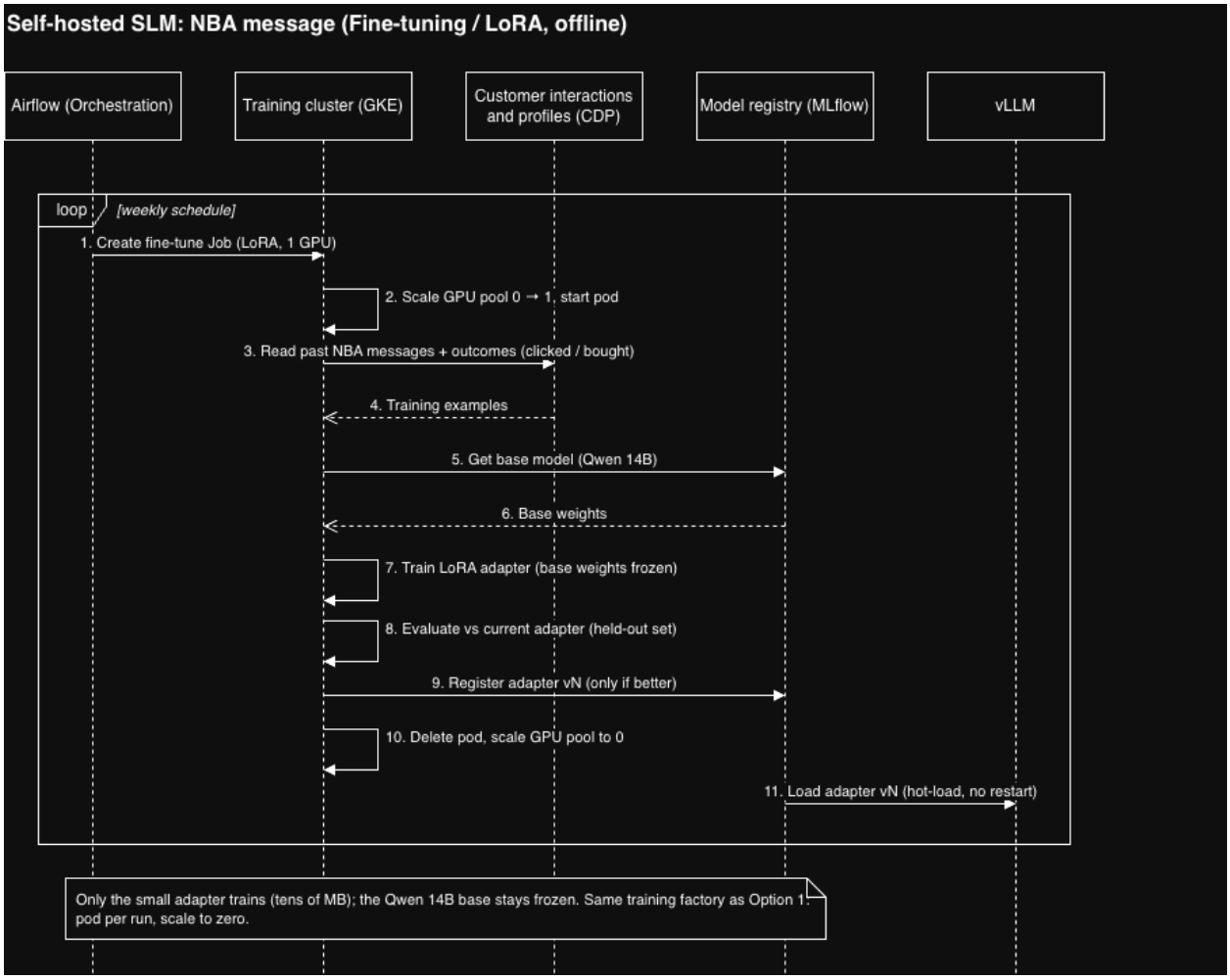

Self-hosted SLM: NBA message (Fine-tuning / LoRA, offline)
Airflow (Orchestration)
Training cluster (GKE)
Customer interactions and profiles (CDP)
Model registry (MLflow)
vLLM
loop
[weekly schedule]
1. Create fine-tune Job (LoRA, 1 GPU)
2. Scale GPU pool 0 → 1, start pod
3. Read past NBA messages + outcomes (clicked / bought)
4. Training examples
5. Get base model (Qwen 14B)
6. Base weights
7. Train LoRA adapter (base weights frozen)
8. Evaluate vs current adapter (held-out set)
9. Register adapter vN (only if better)
10. Delete pod, scale GPU pool to 0
11. Load adapter vN (hot-load, no restart)
Only the small adapter trains (tens of MB); the Qwen 14B base stays frozen. Same training factory as Option 1: pod per run, scale to zero.

Inference

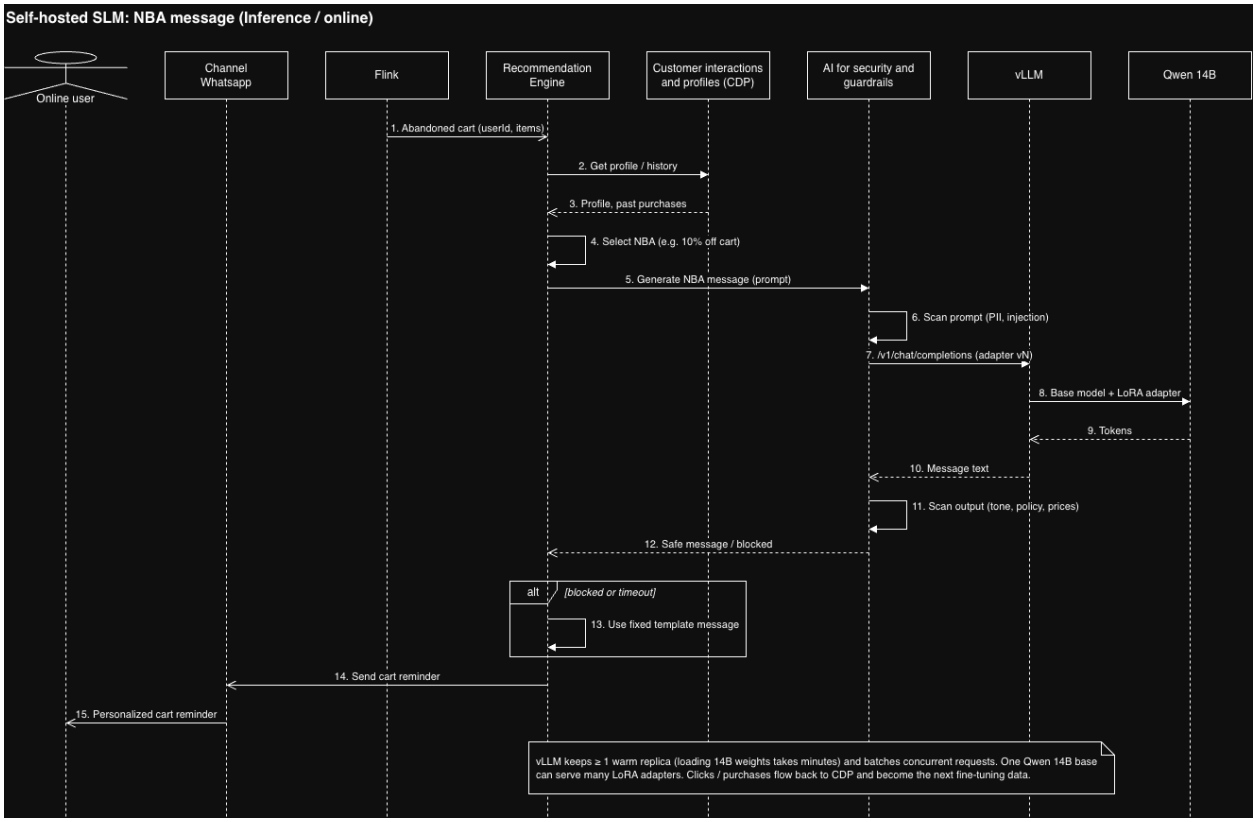


Potential churned customers

Ingestion/Training flow


Self-hosted SLM: NBA message (Fine-tuning / LoRA, offline)
Airflow (Orchestration)
Training cluster (GKE)
Customer interactions and profiles (CDP)
Model registry (MLflow)
vLLM
loop
[weekly schedule]
1. Create fine-tune Job (LoRA, 1 GPU)
2. Scale GPU pool 0 → 1, start pod
3. Read past NBA messages + outcomes (clicked / bought)
4. Training examples
5. Get base model (Qwen 14B)
6. Base weights
7. Train LoRA adapter (base weights frozen)
8. Evaluate vs current adapter (held-out set)
9. Register adapter vN (only if better)
10. Delete pod, scale GPU pool to 0
11. Load adapter vN (hot-load, no restart)
Only the small adapter trains (tens of MB); the Qwen 14B base stays frozen. Same training factory as Option 1: pod per run, scale to zero.

Inference



Customer care Chatbot, supported with RAG

Agentic Marketing - Campaign Automation

Generate new marketing campaign by Agentic AI

Deployment Viewpoint

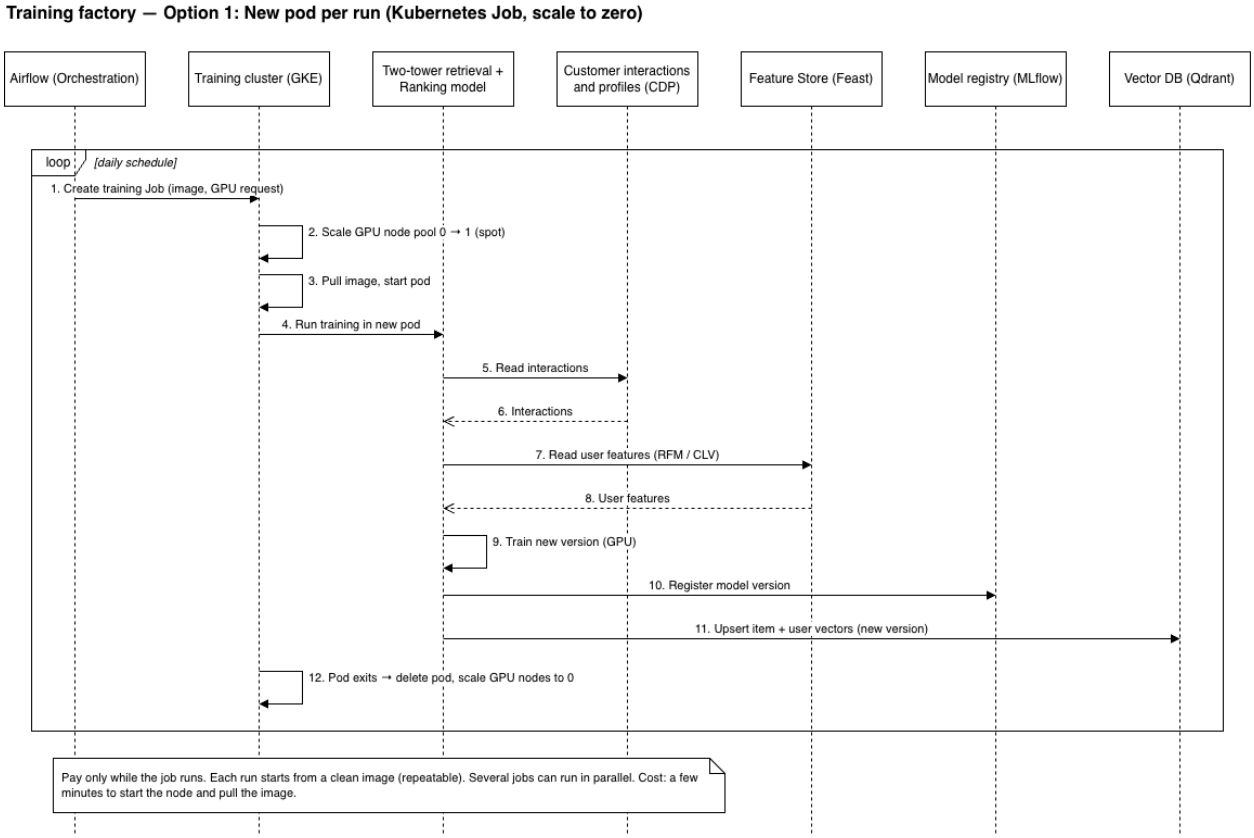
Training Factory - Options compared

Training factory — Options compared

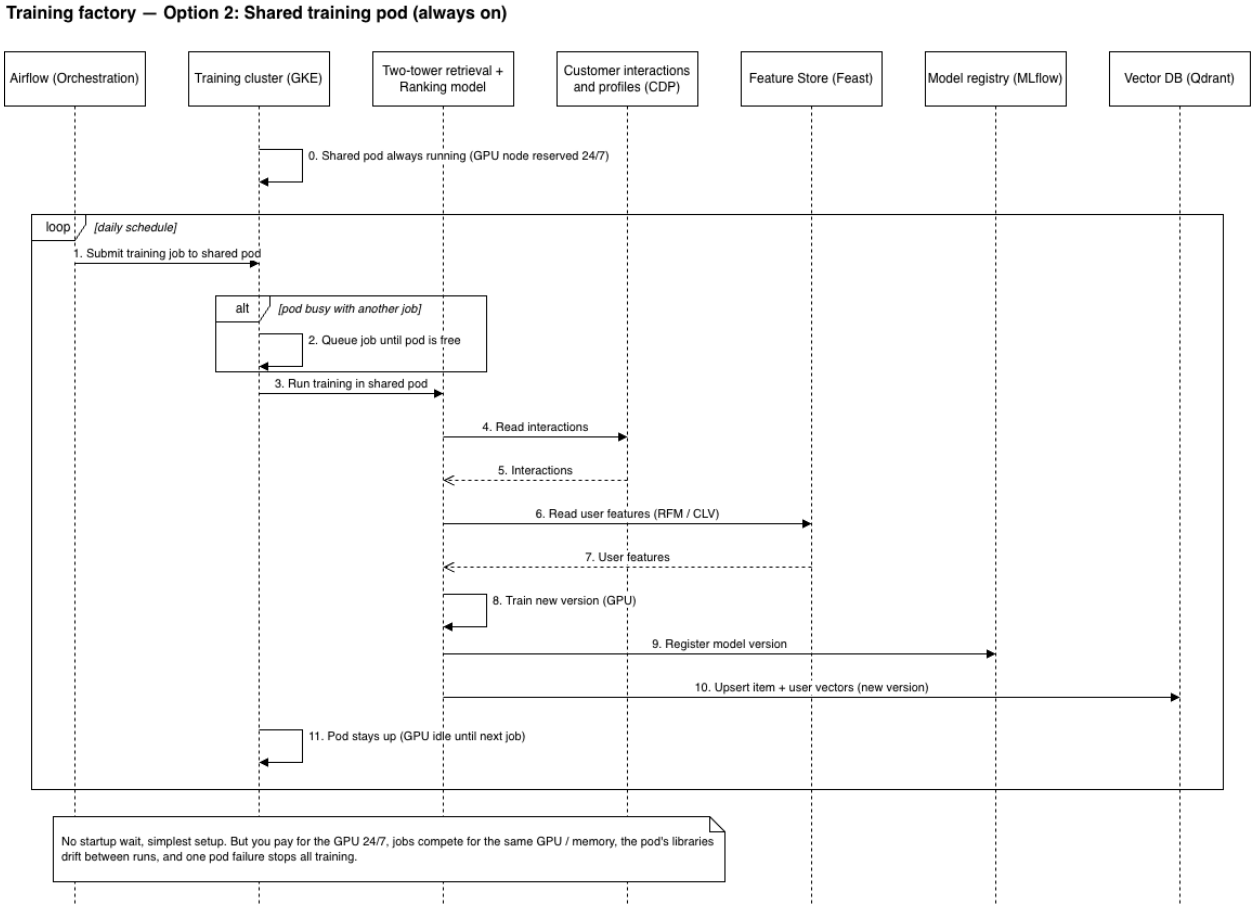
Where do the AI/ML Layer models train? Both options run on the Training cluster (GKE); Airflow triggers, MLflow stores each model version.

	Option 1: New pod per run (recommended)	Option 2: Shared pod, always on
How it works	Airflow creates a pod per run; it trains, then is deleted	One pod always running; Airflow sends jobs to it
Cost	Pay only while training (spot GPUs, scale to zero)	Pay for the GPU 24/7, even when idle
Startup time	Minutes (start node, pull image)	None (pod already warm)
Isolation / repeatability	Clean image every run	Shared state; libraries drift over time
Parallel jobs	Yes, one pod per job	No, jobs queue or compete for GPU
Failure impact	One job fails; next run starts clean	Pod down → all training stops
Ops effort	Needs Kubernetes skills (autoscaling, quotas)	Simple to set up; manual upkeep
Choose when	Batch / scheduled training (our case: daily)	Near-continuous training, or GPUs already paid for (on-prem)

Training factory — Option 1: New pod per run (Kubernetes Job, scale to zero)



Training factory — Option 2: Shared training pod (always on)



Capacity Planning

- For each service
    - 1 Node, 2 pods for every node, 1-core vCPU limit 2 vCPU / 2 GiB, limit 4 GiB.
- API / VPN Gateway added to handle (authentication/auth, rate limiting, routing/discovery, load balancing, observ. E.g. logging, metrics, ..).
- DR very recommended to be there (e.g. main DC is Riyadh, and DR at Al-khobar).

Data Viewpoint

Connections table (TBD)

Source	Destination	Protocol	Port(s)	Authentication	Exposure (pub/private VPN)
Mobile app	E-comerce service	HTTPS (TCP) through gateway	443	JWT, TLS 1.2+	Public with Gateway authen

Data Policy

- Retention policy: all customer's data should be kept for at least 5 years.

API & Data Contracts

Endpoints

Method	**Endpoint**	**Description**
POST/GET	/v1/resource	Create new …

Data mapping / contracts
(Flink/Kafka/…. Data contracts, between two systems or more)...

Operation Viewpoint

Service Level Objectives

Perform load testing on pre-prod for the following

- Availability: 99.8%
- P50 Latency: <100ms
- P95 Latency: 500ms
- Throughput: 1000 RPS

Observability

- Grafana: Utilization metrics, logs and traces, plus dashboards, alerts and incidents.
- LangFuse: AI/ML metrics, llm as a judge, evals, drifting…
- Loki: central logs.
- Prometheus: alerts, send to channels (slack, teams, email, …).

Security Clearance

- Fill in the security survey, and get the security team sign-off.